

AVL Monitor — Guía de instalación y arranque

Prototipo de monitoreo AVL (vehículos en mapa) en **.NET 8 + WPF**, con arquitectura limpia, SQL Server y **Waze Live Map** embebido (WebView2 + iframe oficial).

¿Qué hace el proyecto?

Función	Descripción
Mapa de escritorio	Waze Live Map embebido vía iframe oficial (<code>embed.waze.com</code>) + capa de marcadores AVL
Direcciones	Buscar con Geoapify, guardar en SQL Server, mostrar marcadores
AVL simulado	Vehículos que se mueven en el mapa cada pocos segundos
SQL Server	Tabla <code>Direcciones</code> , SPs, triggers de auditoría
P2L (opcional)	Validación de licencia Google + API Refactorii al arrancar
Signaling (opcional)	Cliente HTTP hacia backend externo de tiempo real

Arquitectura (Clean / Hexagonal)

AvlMonitor.Domain → Entidades puras (Address, Vehicle)
AvlMonitor.Application → Casos de uso y servicios
AvlMonitor.Persistence → Dapper + SQL Server (repositorios, SPs)
AvlMonitor.Infrastructure→ Geoapify, OAuth, P2L, signaling HTTP
AvlMonitor.UI.Wpf → MVVM + WebView2
AvlMonitor.Tests → xUnit + Moq

Equivalencias si vienes de Java/Spring: `DependencyInjection` ≈ `@Configuration` + `@Bean` ;
`servicios de Application` ≈ `@Service` ; `MainViewModel` ≈ controlador de estado de la UI.

Requisitos

Componente	Obligatorio	Notas
Windows 10/11	Sí	WPF solo corre en Windows
.NET 8 SDK	Sí	Descargar
WebView2 Runtime	Sí	Suele venir con Edge; si falla el mapa, instálalo desde Microsoft
SQL Server	Recomendado	Express nativo o Docker (opcional)
Internet	Sí (para el mapa)	Waze Live Map se carga desde <code>embed.waze.com</code> en tiempo real
Geoapify API Key	Opcional	Sin clave no geocodifica, pero mapa Waze y AVL sí funcionan
Docker Desktop	Opcional	Solo si quieres SQL en contenedor; requiere WSL2

Mapa Waze (tiempo real)

El mapa visible es **Waze Live Map** oficial, no OpenStreetMap ni Leaflet como mapa base:

- URL: `https://embed.waze.com/es/iframe` (idioma español)
- Tráfico, incidentes y calles en **tiempo real** desde Waze
- Marcadores de direcciones y unidades AVL se superponen desde la app .NET
- Archivo: `src/AvlMonitor.UI.Wpf/Assets/map.html`

Waze no publica API para pintar marcadores dentro de su mapa. El prototipo usa `iframe` Waze + capa propia de marcadores sincronizada al centrar el mapa desde la app.

Instalación paso a paso

1. Clonar / abrir el proyecto

```
cd "P2L state machine BACK API (1) - Waze FRONT (n) y SQL server DB (n) .NET escritorio
```

2. Instalar .NET 8 SDK

```
winget install Microsoft.DotNet.SDK.8
```

Verificar:

```
dotnet --version  
# Debe mostrar 8.0.x
```

3. Restaurar y compilar

```
dotnet restore .\AvlMonitor.sln  
dotnet build .\src\AvlMonitor.UI.Wpf\AvlMonitor.UI.Wpf.csproj
```

4. Configurar variables locales (opcional)

Copia la plantilla si no existe `.env` :

```
Copy-Item .\src\AvlMonitor.UI.Wpf\.env.example .\src\AvlMonitor.UI.Wpf\.env
```

En desarrollo, deja:

```
P2L_SKIP_LICENSE_GATE=true  
GEOAPIFY_API_KEY=tu_clave_si_la_tienes
```

El build copia automáticamente `.env` (o `.env.example`) a la carpeta de salida del ejecutable.

Licencia P2L desactivada por defecto en `appsettings.json` (`SkipLicenseGate: true`): la app abre directo al mapa sin Google OAuth.

5. Base de datos SQL Server

La app **abre sin SQL** (mapa + vehículos simulados). Para persistir direcciones, elige **una** opción:

Opción A — SQL Server Express (recomendada sin Docker)

1. Descarga [SQL Server 2022 Express](#).
2. Instala con instancia `SQLEXPRESS` y autenticación mixta (o solo Windows).
3. Inicializa la base:

```
.\scripts\setup-dev-db.ps1
```

Cadena por defecto en `appsettings.json` :

```
Server=localhost\SQLEXPRESS;Database=AvlMonitorDb;Trusted_Connection=True;TrustServerCer
```



Opción B — Docker (solo si Docker Desktop funciona)

Docker Desktop en Windows **requiere WSL2**. Si no lo tienes:

```
wsl --install  
# Reinicia el PC, abre Docker Desktop y espera a que arranque
```

Luego:

```
docker compose up -d  
.\scripts\setup-dev-db.ps1
```

Cadena para Docker (descomenta en `.env`):

```
Server=localhost,1433;Database=AvlMonitorDb;User Id=sa;Password=AvlMonitor_Test123!;Trus
```



Scripts SQL (orden manual en SSMS)

Si prefieres ejecutarlos a mano:

1. `database/01_schema.sql`
2. `database/02_stored_procedures.sql`
3. `database/03_triggers.sql`
4. `database/04_seed_dev.sql` (datos de prueba: Bogotá, Medellín, Cali)

Poner el proyecto a correr

Forma rápida (recomendada)

Opción 1 — Doble clic o desde CMD (más fácil):

```
start-dev.cmd
```

Opción 2 — Desde PowerShell:

```
.\scripts\start-dev.ps1
```

Opción 3 — Desde CMD clásico (si escribes solo `start-dev.ps1` , Windows lo abre con Bloc de notas):

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\start-dev.ps1
```

Hace: intenta crear la BD → compila → lanza la app WPF.

Forma manual

```
dotnet run --project .\src\AvlMonitor.UI.Wpf\AvlMonitor.UI.Wpf.csproj
```

Tests unitarios

```
dotnet test .\AvlMonitor.sln
```

Interfaz y demo

Al abrir **AVL Monitor** verás:

- **Izquierda:** **Waze Live Map** en tiempo real (WebView2 + `iframe embed.waze.com`)
- **Derecha:** tabla de direcciones y vehículos AVL
- **Arriba:** buscar dirección, actualizar lista, estado

Flujo de demo:

1. Escribe una dirección (ej. Bogotá, Colombia) → **Buscar y guardar** (requiere Geoapify + SQL).
2. **Actualizar lista** recarga desde SQL Server.
3. Los vehículos simulados se mueven solos en el mapa.
4. Si SQL no está disponible, verás un mensaje en la barra de estado; el mapa y AVL siguen funcionando.

Problemas conocidos y soluciones

Problema	Causa	Solución
PathTooLongException (WebView2)	Ruta del proyecto muy larga en el escritorio	Ya corregido: datos de WebView2 en %LOCALAPPDATA%\AvlMonitor\WebView2
App se cierra al iniciar por SQL	SQL no instalado	Ya corregido: error capturado; la app continúa
Docker no arranca	Falta WSL2	Usa SQL Express (Opción A) o instala WSL y reinicia
Geocoding falla	Sin GEOAPIFY_API_KEY	Regístrate en geoapify.com y pon la clave en .env
Error backend externo	No hay servidor en localhost:7001	Normal en demo; el signaling es opcional

Integración P2L (producción / demo comercial)

1. Panel admin: <https://www.refactorii.com/p2l-tenant/dashboard-wpf>
2. Copia P2L_COMPANY_ID , P2L_COMPANY_KEY , GOOGLE_OAUTH_CLIENT_ID al .env
3. Pon P2L_SKIP_LICENSE_GATE=false
4. Al arrancar: login Google → validación POST /api/p2l-wpf/companies/validate-driver

Estructura de archivos útiles

```
AvlMonitor.sln
database/      → Scripts SQL (schema, SPs, triggers, seed)
scripts/
  start-dev.ps1      → Arranque completo en un comando
  setup-dev-db.ps1   → Solo inicializar BD
docker-compose.yml → SQL Server en Docker (opcional)
tools/DbBootstrap/  → Herramienta .NET para ejecutar scripts SQL
src/AvlMonitor.UI.Wpf/ → App WPF (punto de entrada)
```

Resumen de un comando

```
start-dev.cmd
```

O desde PowerShell:

```
.\scripts\start-dev.ps1
```

La ventana **AVL Monitor · Waze** debería abrirse con mapa Waze en tiempo real y vehículos en movimiento. SQL y Geoapify son opcionales para la demo básica.